Question 1 (2 points)
Consider the grammar:

$$\begin{aligned} S &\rightarrow 0S0 \mid 1S1 \mid 0A1 \mid 1A0 \\ A &\rightarrow 0A \mid 1A \mid \epsilon \end{aligned}$$

What is the language generated by this grammar?

A. Set of all palindromes over $\{0, 1\}$

B. Set of all strings with equal number of 0's and 1's

C. Set of strings of even length

D. Set of all strings over $\{0, 1\}$ that are not palindromes

Option D is the correct answer.

For any string of length n generated by this grammar, there must be at least one i such that the i-th bit and $n-i$-th bit in this string don't match. This is because, in the derivation of this string from the given grammar, one of the production rules $S \rightarrow 0A1$ and $S \rightarrow 1A0$ must be used exactly once. If the production rule $S \rightarrow 0A1$ or $S \rightarrow 1A0$ occurs as the i-th production rule in the derivation, then we are guaranteed that the i-th bit and $n-i$-th bit of the string must be different.

Similarly, we can argue that any string that is not a palindrome can be generated by this grammar. Let us consider a string x of length n that is not a palindrome. There must be some i such that the i-th bit and $n-i$-th bit of x are different. Let us consider the smallest such i. Now, in order to generate the first and last $i-1$ bits of x, we can use the production rules $S \rightarrow 0S0$ or $S \rightarrow 1S1$. For the i-th production rule in the derivation, we have to use either $S \rightarrow 0A1$ or $S \rightarrow 1A0$. Now, we can use the production rules $A \rightarrow 0A$, $A \rightarrow 1A$ and $A \rightarrow \epsilon$ to generate the remaining bits of x (sandwiched between the i-th and $n-i$-th bits).

Question 2 (2 points)
Which of the following regular expressions defines the same language as the CFG:

$$S \rightarrow A \mid \epsilon, \quad A \rightarrow 01A \mid 01B, \quad B \rightarrow 01B \mid \epsilon$$

A. $(10)^*$

B. $(01)^*$

C. 0^*1^*

D. $0^* + 1^*$

Option B is the correct answer.

The given grammar is actually equivalent to the grammar

$$S \rightarrow 01S \mid \epsilon$$

. The symbols A and B are redundant. Now, it is easy to see that this reduced grammar is equivalent to the regular expression $(0, 1)^*$

Question 3 (2 points)
Consider the following context-free grammar (CFG):

$$S \rightarrow 1A0 \mid 0A1, \quad A \rightarrow 0A0 \mid 1A1 \mid \epsilon$$

Which of the following strings belongs to the language described by the above CFG?

A. 1001

B. 0110

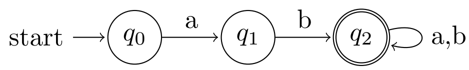
C. 1010

D. 001100

There is a typo in the question. None of the given options belong to the language generated by the given grammar. This question will not be considered for evaluation.

Question 4 (2 points)

Consider the following DFA:



Which of the following CFGs generates the same language as the given DFA?

A.

$$
\begin{aligned}
S &\to aA \\
A &\to bB \\
B &\to aB \mid bB \mid \epsilon
\end{aligned}
$$

B.

$$
\begin{aligned}
S &\to bA \\
A &\to aB \\
B &\to aB \mid bB \mid \epsilon
\end{aligned}
$$

C.

$$S \to aS \mid bS \mid \epsilon$$

D.

$$S \to abS \mid \epsilon$$

Option A is the correct answer.

There is a typo in the question. The given diagram is of an NFA and not a DFA. However, the question still remains valid. Using the method given in Section 8.1.4 of Week 4 Lecture notes, we can derive a context-free grammar for the given NFA (by converting the NFA to a DFA first and eliminating the variables that correspond to dead states in the DFA).

Question 5 (2 points)

Which CFG generates the same language as the regular expression $(aba)^*$?

A. $S \to abaS \mid \epsilon$

B. $S \to aSbSa \mid \epsilon$

C. $S \to aSa \mid bSb \mid \epsilon$

D. $S \to abS \mid baS \mid \epsilon$

Option A is correct.

Question 6 (2 points)

Which of the following grammars is not ambiguous?

A. $S \to 0S \mid 1S \mid S0 \mid S1 \mid \epsilon$

B. $S \to SS \mid 0 \mid 1$

C. $S \to 0S0 \mid 1S1 \mid \epsilon$

D. $S \to A \mid B, \quad A \to 0A \mid \epsilon, \quad B \to 1B \mid \epsilon$

Option C is correct.

This grammar corresponds to the set of all binary even length palindromes. Each string can in the set can be generated using a unique sequence of production rules. If the string s (belonging to the set of binary even length palindromes) is of length n, then the i-th production rule in the derivation of s is:

1) $S \to 0S0$, if $i \leq n/2$ and the i-th bit in s is 0. 2) $S \to 1S1$, if $i \leq n/2$ and the i-th bit in s is 1. 3) $S \to \epsilon$, if $i = n/2$.

Since there is a unique derivation for any string in the language, therefore this grammar is unambiguous.

The other three given grammars are not unambiguous.

A) Consider the string 0. It can be derived in two ways- $S \to S0 \to 0$ or $S \to 0S \to 0$

B) Consider the string 000. It can be derived in multiple ways- $S \to SS \to 0S \to 0SS \to 00S \to 000$ or $S \to SS \to S0 \to SS0 \to S00 \to 000$.

D) Consider the string ϵ. It can be derived in two ways- $S \to A \to \epsilon$ or $S \to B \to \epsilon$.

Question 7 (2 points)

Let a DFA D with n states accept a regular language L. What is the minimum number of non-terminals required in a context-free grammar G that generates L?

A. n

B. $2n$

C. n^2

D. 2^n

Option A is the correct answer.

Refer to Section 8.1.4 of Week 4 Lecture notes. We can construct a grammar with n non-terminals for a DFA with n states. (where each non-terminal in the grammar corresponds to a state in the DFA)

Question 8 (2 points)

Which of the following CFGs generates all palindromes over $\{a, b\}$?

A. $S \to aSa \mid bSb \mid a \mid b \mid \epsilon$

B. $S \to aSb \mid bSa \mid \epsilon$

C. $S \to aS \mid bS \mid \epsilon$

D. $S \to aaS \mid bbS \mid a \mid b$

Option A is the correct answer. Palindromes are strings which when reversed remains unchanged. Note that for such strings, the last and the first character must be same. Also if we remove the first and last character from the string, the remaining string is also a palindrome. Using this idea, we construct the grammar as given in option B.

Note:- The production rules $S \longrightarrow a$ and $S \longrightarrow b$ are used to generate odd-length palindromes (eg.- aba) where there is a single middle character. On the other hand, the production rule $S \longrightarrow \epsilon$ is used to generate even-length palindromes which don't have a single middle character.